

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

06 - Árvores binárias de busca e AVL

Prof. Alex Kutzke

Setembro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Enunciar a propriedade da árvore binária de busca e verificar se uma árvore dada a satisfaz;
2. Implementar busca, inserção, mínimo e máximo em Go, e justificar por que o custo das quatro operações é a altura da árvore;
3. Executar à mão a remoção nos três casos;
4. Explicar por que a ordem de inserção determina a forma da árvore;
5. Calcular o fator de balanceamento e identificar qual dos quatro casos de desbalanceamento ocorreu;
6. Aplicar rotações simples e duplas;
7. Comparar os custos de ABB e AVL no melhor caso, no caso médio e no pior caso.

Este conteúdo cai na Prova 1.

Roteiro

Motivação e definição

Operações

Forma e desempenho

Balanceamento AVL

Rotações

Custos

Guardar um conjunto de chaves

Duas operações: inserir uma chave nova, e responder se uma chave está no conjunto.

Estrutura	Inserção	Busca (pior caso)
Slice sem ordem	$O(1)$ amortizado	$\Theta(n)$
Slice ordenado	$\Theta(n)$	$\Theta(\log n)$ busca binária

No slice ordenado, inserir no meio obriga a empurrar para a direita, uma a uma, todas as chaves que vêm depois da chave nova.

A medição: 200 mil inserções, 10 mil buscas

```
$ go run motivacao.go
200000 chaves inseridas, 10000 buscas
```

estrutura	inserção	busca
-----	+-----	+-----
slice sem ordem	3ms	325ms
slice ordenado	2.334s	1ms
árvore de busca	54ms	2ms

A árvore de busca fica perto do melhor das duas colunas. Ela não precisa manter as chaves contíguas na memória: a ordem está na forma da árvore, e mudar a forma custa alterar um ponteiro.

A busca que a aula anterior deixou em aberto

```
func Pertence(a *No, procurado int) bool {  
    if a == nil {  
        return false  
    }  
    if a.Info == procurado {  
        return true  
    }  
    return Pertence(a.Esq, procurado) ||  
        Pertence(a.Dir, procurado)  
}
```

Visita todos os nós no pior caso: nada na árvore diz de que lado continuar.

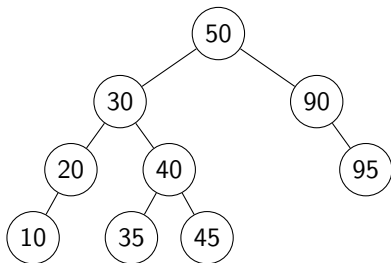
Mas o in-ordem da árvore de exemplo saiu crescente: 10 20 30 35 40 45 50 90 95. Aquela árvore tinha sido montada com os menores à esquerda e os maiores à direita.

Árvore binária de busca

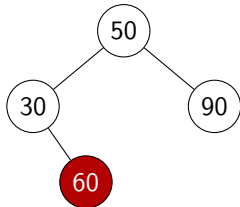
Árvore binária em que, para **todo** nó x :

- as chaves da subárvore esquerda de x são menores que a chave de x ;
- as chaves da subárvore direita de x são maiores que a chave de x .

Nesta aula, chaves distintas.



A exigência vale para todos os nós



Cada nó está do lado certo em relação ao **seu pai**, e ainda assim esta árvore **não é uma ABB**: o 60 está na subárvore esquerda do 50.

O erro clássico na função que verifica a propriedade é comparar cada nó apenas com os seus dois filhos. Esse teste aceita a árvore ao lado.

Duas consequências da definição

O percurso in-ordem devolve as chaves em ordem crescente

A garantia que a árvore da aula anterior tinha por construção agora é obrigatória. É o teste mais rápido para saber se uma operação preservou a propriedade de busca.

A busca nunca precisa voltar atrás

Em cada nó, a comparação descarta uma subárvore inteira. O caminho percorrido é único, e tem no máximo a altura da árvore mais um nó.

Busca

```
func Busca(a *No, procurado int) bool {  
    if a == nil {  
        return false  
    }  
    if procurado < a.Info {  
        return Busca(a.Esq, procurado)  
    }  
    if procurado > a.Info {  
        return Busca(a.Dir, procurado)  
    }  
    return true  
}
```

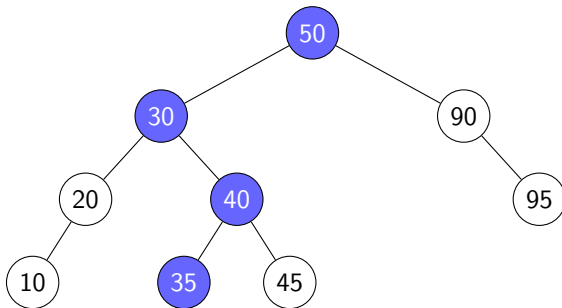
Três resultados na comparação, e cada um decide o passo seguinte.

A mesma busca, sem recursão

```
func BuscaIterativa(a *No, procurado int) bool {  
    for a != nil {  
        if procurado < a.Info {  
            a = a.Esq  
        } else if procurado > a.Info {  
            a = a.Dir  
        } else {  
            return true  
        }  
    }  
    return false  
}
```

As chamadas recursivas estavam na posição final e nenhuma precisava do resultado da outra. A *Pertence* do slide anterior precisa das duas subárvores, e não vira laço com a mesma facilidade.

O caminho da busca por 35



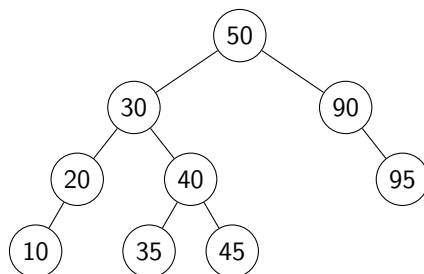
Quatro comparações, quatro nós visitados de nove. Em cada nó, metade do que restava é descartada sem ser olhada.

Inserção

```
func Insere(a *No, v int) *No {  
    if a == nil {  
        return &No{Info: v}  
    }  
    if v < a.Info {  
        a.Esq = Insere(a.Esq, v)  
    } else if v > a.Info {  
        a.Dir = Insere(a.Dir, v)  
    }  
    return a  
}
```

O nó novo entra como **folha**, na posição em que a busca por aquela chave terminaria. Quem chama escreve `a = Insere(a, v)`, o que cobre também a árvore vazia.

Onde entra o 37

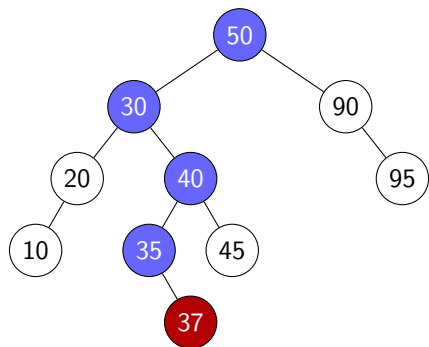


$a = \text{Insere}(a, 37)$

De qual nó o 37 vira filho, e de que lado?

Quantas comparações até chegar lá?

Onde entra o 37



$37 < 50$: esquerda.

$37 > 30$: direita.

$37 < 40$: esquerda.

$37 > 35$: direita, e ali está o nil.

A linha `a.Esq =`

`Inser(a.Esq, v)` reatribui o ponteiro em todos os nós do caminho, embora só um deles mude. É o preço de não guardar o ponteiro para o pai.

Mínimo, máximo e listagem em ordem

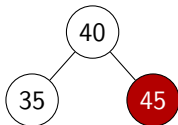
```
func Minimo(a *No) (int, bool) {  
    if a == nil {  
        return 0, false  
    }  
    for a.Esq != nil {  
        a = a.Esq  
    }  
    return a.Info, true  
}
```

O menor valor está no nó mais à esquerda, o maior no mais à direita. Nenhuma comparação de chaves é necessária.

O segundo valor devolvido diz se a resposta existe. A árvore vazia não tem mínimo, e devolver 0 sozinho a confundiria com uma árvore que contém o 0.

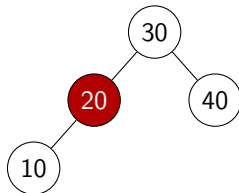
Remoção: dois casos fáceis

Caso 1: folha



O nó some, e o pai passa a apontar para nil.

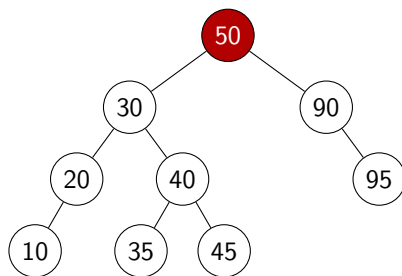
Caso 2: uma subárvore só



O filho sobe para o lugar do nó removido, com toda a sua subárvore.

No caso 2 a propriedade continua valendo: tudo o que subiu já estava do lado certo em relação ao avô.

Remoção: duas subárvores



$a = \text{Remove}(a, 50)$

Nenhum dos dois filhos pode simplesmente subir.

Que chave da árvore pode ocupar o lugar do 50? Quantas servem?

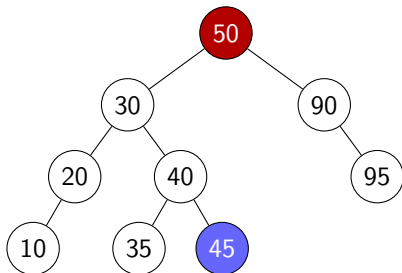
Remoção: duas subárvores

O lugar do 50 comporta uma chave maior que tudo à esquerda e menor que tudo à direita. Existem duas assim:

- o **antecessor**, maior valor da subárvore esquerda (45);
- o **sucessor**, menor valor da subárvore direita (90).

Copiamos uma das duas para o nó e removemos o nó de onde ela veio.

O antecessor não tem filho à direita, então essa segunda remoção cai no caso 1 ou no caso 2. **Nunca no caso 3.**



Remoção: achar o nó

```
func Remove(a *No, v int) *No {  
    if a == nil {  
        return nil  
    }  
    if v < a.Info {  
        a.Esq = Remove(a.Esq, v)  
        return a  
    }  
    if v > a.Info {  
        a.Dir = Remove(a.Dir, v)  
        return a  
    }  
    // daqui em diante: a.Info == v, o nó a remover é este
```

A descida é a mesma da busca. Cada chamada devolve a subárvore já corrigida, e o pai a reatribui em `a.Esq` ou `a.Dir`. Quem chama de fora escreve `a = Remove(a, v)`.

Remoção: os três casos

```
// continuação de Remove, com a.Info == v
if a.Esq == nil && a.Dir == nil {
    return nil
}
if a.Esq == nil {
    return a.Dir
}
if a.Dir == nil {
    return a.Esq
}

antecessor := a.Esq
for antecessor.Dir != nil {
    antecessor = antecessor.Dir
}
a.Info = antecessor.Info
a.Esq = Remove(a.Esq, antecessor.Info)
return a
}
```

Remoção: as três saídas

Remove(a, 45), caso 1: folha

in-ordem: 10 20 30 35 40 50 90 95

Remove(a, 20), caso 2: uma subárvore só

in-ordem: 10 30 35 40 50 90 95

Remove(a, 50), caso 3: duas subárvores (a raiz)

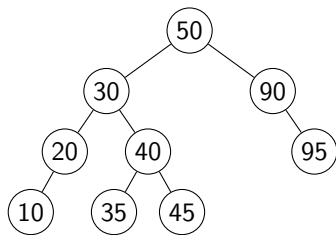
```
      95
     /
    90
   /
  40
 /
35
/
30
/
10
```

in-ordem: 10 30 35 40 90 95

O 40, maior valor da subárvore esquerda, tomou o lugar do 50 na raiz. O in-ordem continua crescente em todas as saídas.

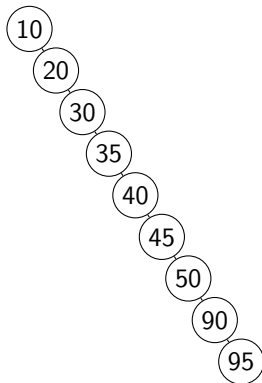
A ordem de inserção determina a forma

50 30 90 20 40 95 10 35 45



altura 3

10 20 30 35 40 45 50 90 95



altura 8

Mesmos nove valores. A segunda é uma lista encadeada com passos extras.

Entrada ordenada é comum

Chaves lidas de um arquivo já ordenado, códigos sequenciais e carimbos de tempo produzem a árvore degenerada, justamente nos casos em que a árvore seria mais útil.

Entre os dois extremos está o caso médio, com as chaves chegando em ordem aleatória:

n	mínima	aleatória	degenerada	comparações médias
1.000	9	21,1	999	12,0
10.000	13	30,6	9.999	16,6
100.000	16	40,6	99.999	21,0
1.000.000	19	46,8	999.999	25,3

O número médio de comparações de uma busca bem sucedida tende a $2 \ln n \approx 1,39 \log_2 n$, cerca de 39% acima da árvore perfeitamente balanceada.

O custo de todas as operações é a altura

Busca, inserção, remoção, mínimo e máximo percorrem um único caminho da raiz para baixo: $O(h)$.

Situação	Altura	Custo
Árvore cheia (melhor caso)	$\log_2(n + 1) - 1$	$\Theta(\log n)$
Ordem aleatória (caso médio)	$\Theta(\log n)$	$\Theta(\log n)$
Ordem crescente (pior caso)	$n - 1$	$\Theta(n)$

O caso médio supõe que a ordem de chegada das chaves seja aleatória, e quem escreve a estrutura não controla essa ordem.

O critério AVL

Manter a árvore perfeitamente balanceada custaria caro: uma única inserção pode obrigar a reconstruir quase tudo. Adelson-Velsky e Landis, em 1962, exigiram menos.

Árvore AVL

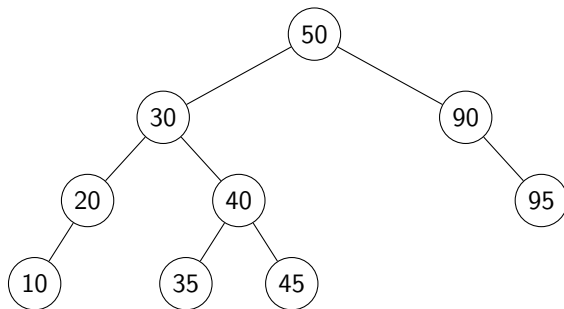
Árvore binária de busca em que, para todo nó, as alturas das duas subárvores diferem em no máximo 1.

Fator de balanceamento de um nó:

$$fb(x) = altura(x.Esq) - altura(x.Dir)$$

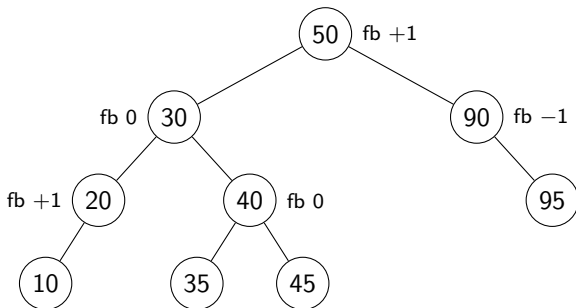
Em uma AVL, todo nó tem fb igual a -1 , 0 ou $+1$. Com $fb = +2$, o nó está pesado à esquerda; com $fb = -2$, à direita. A árvore vazia tem altura -1 .

Fator de balanceamento na árvore de exemplo



Qual é o fator de balanceamento de cada um dos cinco nós internos?
A árvore é AVL?

Fator de balanceamento na árvore de exemplo



Todos os fatores estão em $\{-1, 0, +1\}$: a árvore de exemplo é AVL.
A degenerada do slide anterior não é, a raiz dela tem $\text{fb} = -8$.

A altura guardada no nó

```
type No struct {  
    Info int  
    Alt  int  
    Esq  *No  
    Dir  *No  
}  
  
func altura(a *No) int {  
    if a == nil {  
        return -1  
    }  
    return a.Alt  
}  
  
func atualizaAltura(a *No) {  
    a.Alt = 1 + maior(altura(a.Esq), altura(a.Dir))  
}
```

Recalcular a altura a cada verificação custaria $\Theta(n)$. Com o campo `Alt`, calcular o fator de balanceamento custa tempo constante.

Por que o critério fraco basta

Seja $N(h)$ o **menor** número de nós de uma AVL de altura h . A árvore mais esguia possível tem uma raiz, uma subárvore de altura $h - 1$ e outra tão pequena quanto o critério permite, de altura $h - 2$:

$$N(h) = N(h - 1) + N(h - 2) + 1, \quad N(0) = 1, \quad N(1) = 2$$

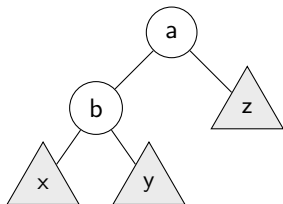
h	0	1	2	3	4	5	6	7	8	9	10
$N(h)$	1	2	4	7	12	20	33	54	88	143	232

É a recorrência de Fibonacci deslocada, $N(h) = F(h + 3) - 1$, e cresce exponencialmente com h . Invertendo:

$$h < 1,44 \log_2(n + 2)$$

Uma AVL com 10 mil chaves tem altura no máximo 17: a menor AVL de altura 18 já precisaria de 10.945 nós.

Rotação à direita



A subárvore está pesada à esquerda.
Queremos que **b** suba para a raiz e **a** desça para a direita.

Onde ficam *x*, *y* e *z* depois disso?

Só uma das três subárvores muda de pai. Qual, e por quê?

Rotação à direita



In-ordem da árvore da esquerda: x, b, y, a, z .

In-ordem da árvore da direita: x, b, y, a, z .

As duas sequências são iguais, e por isso a rotação nunca viola a propriedade de busca. O que muda é a altura: b sobe um nível com x , a desce um nível com z .

Rotação à direita em Go

```
func RotacaoDireita(a *No) *No {  
    b := a.Esq  
    a.Esq = b.Dir  
    b.Dir = a  
  
    atualizaAltura(a)  
    atualizaAltura(b)  
    return b  
}
```

Três atribuições de ponteiro, e a ordem entre elas importa: `b.Dir` é lido para dentro de `a.Esq` antes de ser sobrescrito com `a`.

As alturas são atualizadas de baixo para cima. A função devolve a nova raiz da subárvore, e quem chamou reatribui o ponteiro. `RotacaoEsquerda` é a imagem espelhada.

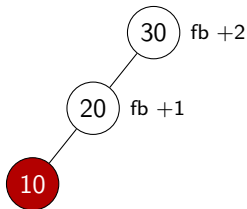
Os quatro casos

Depois de uma inserção, o primeiro nó desbalanceado na volta da recursão tem $fb = +2$ ou $fb = -2$. A rotação a aplicar depende também do lado em que o filho pesado cresceu.

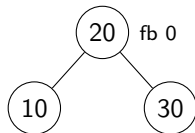
Caso	fb do nó	fb do filho	Correção
Esquerda-esquerda	+2	+1	simples à direita
Direita-direita	-2	-1	simples à esquerda
Esquerda-direita	+2	-1	esquerda no filho, direita no nó
Direita-esquerda	-2	+1	direita no filho, esquerda no nó

Nos dois primeiros casos o nó e o filho pendem para o mesmo lado, e uma rotação resolve. Nos dois últimos o desequilíbrio faz um zigue-zague, e uma rotação simples apenas o inverteria.

Caso esquerda-esquerda: inserir 30, 20, 10

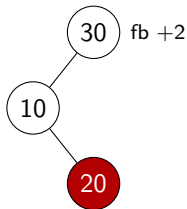


rotação
à direita



Uma rotação simples à direita no 30. Altura 2 antes, altura 1 depois.

Caso esquerda-direita: inserir 30, 10, 20

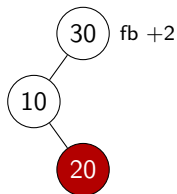


O 30 está pesado à esquerda, como no caso anterior.

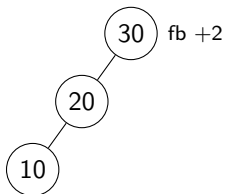
Uma rotação simples à direita no 30 resolve?

Compare o fator do 30 com o do filho 10. No caso anterior os dois tinham o mesmo sinal.

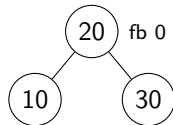
Caso esquerda-direita: inserir 30, 10, 20



à esquerda
no 10



à direita
no 30



O 30 tem $fb +2$ e o filho 10 tem $fb -1$, sinais contrários: é o zigue-zague. Rotação à esquerda no filho, que o alinha, e rotação à direita no nó.

Rebalancear

```
func rebalanceia(a *No) *No {  
    fb := fator(a)  
  
    if fb > 1 { // pesada à esquerda  
        if fator(a.Esq) < 0 { // caso esquerda-direita  
            a.Esq = RotacaoEsquerda(a.Esq)  
        }  
        return RotacaoDireita(a)  
    }  
  
    if fb < -1 { // pesada à direita  
        if fator(a.Dir) > 0 { // caso direita-esquerda  
            a.Dir = RotacaoDireita(a.Dir)  
        }  
        return RotacaoEsquerda(a)  
    }  
  
    return a  
}
```

Inserção em AVL

```
func Insere(a *No, v int) *No {  
    if a == nil {  
        return &No{Info: v}  
    }  
  
    if v < a.Info {  
        a.Esq = Insere(a.Esq, v)  
    } else if v > a.Info {  
        a.Dir = Insere(a.Dir, v)  
    } else {  
        return a  
    }  
  
    atualizaAltura(a)  
    return rebalanceia(a)  
}
```

A inserção da ABB com duas linhas a mais, executadas na volta da recursão em cada nó do caminho da folha nova até a raiz.

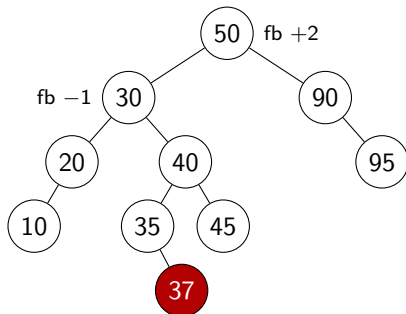
Uma rotação por inserção

Uma inserção estraga o balanceamento de no máximo um nó. A rotação, simples ou dupla, devolve à subárvore a altura que ela tinha **antes** da inserção.

Como a altura da subárvore não mudou, nenhum nó acima dela precisa ser tocado: a inserção em AVL faz **no máximo uma** rotação.

A remoção não tem essa propriedade. Ela pode exigir rotações em todos os nós do caminho até a raiz, e é escrita do mesmo jeito: a remoção da ABB, seguida de `atualizaAltura` e `rebalanceia`.

O 37, de novo



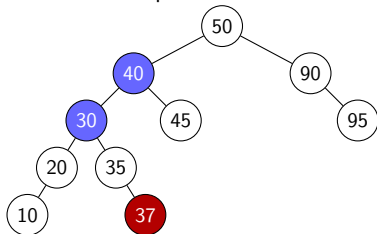
A árvore de exemplo era AVL. Inserimos o 37 como folha, e ela deixou de ser.

Qual nó está desbalanceado?

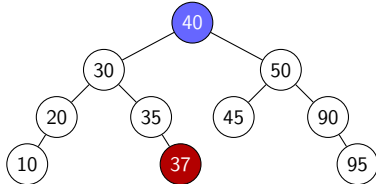
Qual dos quatro casos ocorre?

O 37: rotação dupla na raiz

à esquerda no 30



e à direita no 50



Raiz com fb +2 e filho pesado com fb -1: caso esquerda-direita.

A árvore volta à altura 3, e o in-ordem continua 10 20 30 35 37 40 45 50 90 95.

Entrada ordenada nas duas estruturas

Inserindo 1, 2, 3, ..., n em ordem crescente:

n		altura na ABB		altura na AVL
-----+-----+-----				
10		9		3
100		99		6
1000		999		9
10000		9999		13

Com 10 mil chaves em ordem crescente, a busca na ABB percorre até 10 mil nós, e na AVL até 14. A altura 13 é a menor possível para 10 mil nós.

ABB e AVL

	ABB	AVL
Operações no caso médio	$\Theta(\log n)$	$\Theta(\log n)$
Operações no pior caso	$\Theta(n)$	$\Theta(\log n)$
Memória por nó	chave e 2 ponteiros	chave, 2 ponteiros e altura
Rotações por inserção	nenhuma	no máximo 1
Rotações por remoção	nenhuma	até $O(\log n)$

As árvores rubro-negras, na segunda metade da disciplina, admitem árvores um pouco mais desbalanceadas e fazem menos rotações na remoção, o que as torna a escolha mais comum em bibliotecas de uso geral. O critério de decisão é a proporção entre buscas e alterações na aplicação.

De volta à medição da abertura

O slice ordenado é rápido na busca porque as chaves estão em ordem, e lento na inserção porque manter a ordem exige mover as chaves.

A árvore de busca guarda a mesma informação de ordem na forma da árvore, e por isso inserir custa descer até uma folha e pendurar um nó.

Aquela medição usou chaves em ordem aleatória, que é o caso médio. Com as mesmas 200 mil chaves em ordem crescente, a ABB daria uma coluna de 200 mil níveis, e as duas medições ficariam piores que as do slice.

A AVL remove a dependência da ordem de chegada.

Exercícios

Os exercícios estão no roteiro da aula, na página do material. Eles **não valem nota** e não têm entrega: preparam a aula prática de árvores, essa sim avaliada, e o conteúdo cai na Prova 1.

Códigos da aula, para executar com `go run`:

- `06/codes/motivacao.go`: a medição da abertura;
- `06/codes/abb.go`: busca, inserção, mínimo e máximo;
- `06/codes/remocao.go`: os três casos da remoção;
- `06/codes/forma.go`: ordem de inserção e altura;
- `06/codes/avl.go`: rotações, os quatro casos e a comparação com a ABB.